

Technische Universität Berlin

Faculty of Electrical Engineering and Computer Science
Dept. of Computer Engineering and Microelectronics
Remote Sensing Image Analysis Group



Analysis of Merging Strategies for Continual Parameter-Efficient Adaptation of Foundation Models for Earth Observation

Bachelor of Science in Computer Science

Nov, 2025

Theodor Hagen

Matriculation Number: 457864

Supervisor: Prof. Dr. Begüm Demir

Advisor: Lars Möllenbrok

Eidesstattliche Erklärung

Hiermit erkläre ich an Eides statt, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe angefertigt habe. Sämtliche benutzten Informationsquellen sowie das Gedankengut Dritter wurden im Text als solche kenntlich gemacht und im Literaturverzeichnis angeführt. Die Arbeit wurde bisher nicht veröffentlicht und keiner Prüfungsbehörde vorgelegt.

Hereby I declare that I wrote this thesis myself with the help of no more than the mentioned literature and auxiliary means.

Berlin, 22.10.2025

.....

Theodor Hagen

Acknowledgements

I would like to express my sincere gratitude to Lars Möllenbrok, who provided continuous guidance on my work, offered prompt advice whenever needed, and gave invaluable assistance whenever problems arose.

I am also deeply thankful to Prof. Dr. Begüm Demir for granting me the opportunity to write my thesis with the Remote Sensing Image Analysis Group.

Abstract

Sequentially trained neural networks suffer from catastrophic forgetting, a phenomenon characterized by the loss of previously acquired knowledge when learning new information. This is particularly challenging in the field of Earth Observation, where vast amounts of new sensor data, are generated on a daily basis, requiring models to adapt continuously. As a solution, Continual Learning (CL) – a paradigm that enables models to learn from a stream of data while retaining previously acquired knowledge – has emerged. This thesis investigates the use of parameter-efficient Low-Rank-Adaptation (LoRA) as a CL strategy to mitigate catastrophic forgetting. To enable this, a pre-trained foundation model is utilized for providing a robust knowledge base. The approach involves training task-specific LoRA adapters for new domains while keeping the core foundation model’s parameters frozen. Subsequently, these adapters are combined using three merging techniques – Weight Averaging (WA), which naively averages the LoRA parameters of task-specific adapters, TIES-Merging, which adapts WA by also reducing parameter interference through the pruning of low-magnitude weights and resolving sign conflicts, as well as Learnable Concatenation (CAT), which concatenates the adapters and then learns per-adapter coefficients on a small calibration set, to merge them.

This research was conducted within a Domain-Incremental Learning framework for multi-label land cover classification, utilizing the BigEarthNet v2.0. A systematic comparison of the three adapter-merging strategies shows that all evaluated merging methods reduce catastrophic forgetting relative to a standard sequential training baseline, improving Average mAP($\uparrow$) from 0.639 to 0.717/0.728/0.777 and reducing Forgetting($\downarrow$) from 0.241 to 0.090/0.050/0.038 (WA/TIES/CAT). While the CAT strategy yielded the highest performance, its exponential parameter growth limits its practical application in scenarios with many sequential tasks. In contrast, an adapted variant, LinCAT, which ensures linear parameter growth, offers a favorable trade-off. It approaches the performance of CAT while maintaining scalability. The findings demonstrate that merging LoRA adapters is a practical strategy for the continual, parameter-efficient adaptation of foundation models in the field of Earth Observation.

Zusammenfassung

Sequenziell trainierte neuronale Netze leiden unter katastrophalem Vergessen, einem Phänomen, das durch den Verlust zuvor erworbenen Wissens beim Lernen neuer Informationen gekennzeichnet ist. Dieser Effekt ist insbesondere im Bereich der Erdbeobachtung bemerkbar, wo täglich enorme Mengen neuer Sensordaten erzeugt werden, die eine kontinuierliche Anpassung der Modelle erfordern. Als Lösungsansatz hat sich Continual Learning (CL) etabliert – ein Paradigma, welches Modellen ermöglicht, aus einem Strom neuer Daten zu lernen und dabei zuvor erworbenes Wissen zu bewahren. Diese Arbeit untersucht den Einsatz der parameter-effizienten Low-Rank-Adaptation (LoRA) als CL-Strategie zur Abschwächung katastrophalen Vergessens. Hierfür wird ein vortrainiertes Foundation Model verwendet, welches eine robuste Wissensbasis bereitstellt. Der Ansatz besteht darin, für neue Domänen aufgabenspezifische LoRA-Adapter zu trainieren, während die Parameter des zugrunde liegenden Foundation Models eingefroren bleiben. Anschließend werden diese Adapter mit drei Merging-Techniken kombiniert: Weight Averaging (WA) mittelt die LoRA-Parameter der aufgabenspezifischen Adapter naiv. TIES-Merging erweitert WA, unter anderem durch das Herausfiltern von Gewichten geringer Magnitude und das Auflösen von Vorzeichenkonflikten. Sowie Learnable Concatenation (CAT), das die Adapter konkateniert und anschließend auf einem kleinen Kalibrierungsdatensatz adapter-spezifische Koeffizienten lernt, um sie zu mergen.

Diese Arbeit wurde in einem Domain-Incremental-Learning-Framework für die Multi-Label-Landbedeckungsklassifikation unter Verwendung von BigEarthNet v2.0 durchgeführt. Ein systematischer Vergleich der drei Adapter-Merging-Strategien zeigt, dass alle betrachteten Merging-Methoden das katastrophale Vergessen im Vergleich zu einer Standard-Baseline mit sequenziellem Training reduzieren. Dies lässt sich empirisch durch eine Erhöhung der Average mAP ($\uparrow$) von 0,639 auf 0,717/0,728/0,777 und durch eine Reduzierung der Forgetting-Metrik ($\downarrow$) von 0,241 auf 0,090/0,050/0,038 (WA/TIES/CAT) zeigen. Während die CAT-Strategie die höchste Performance erzielte, schränkt ihr exponentielles Parameterwachstum die praktische Anwendbarkeit in Szenarien mit vielen sequentiellen Tasks ein. Im Gegensatz dazu bietet eine angepasste Variante, LinCAT, die lineares Parameterwachstum sicherstellt, einen vorteilhaften Trade-off. Sie nähert sich der Performance von CAT an, behält dabei jedoch die Skalierbarkeit bei. Die Ergebnisse zeigen, dass das Mergen von LoRA-Adaptoren eine praktische Strategie für die kontinuierliche, parameter-effiziente Anpassung von Foundation Models im Bereich der Erdbeobachtung darstellt.

Contents

List of Acronyms	vii
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Research Objective	2
1.3 Thesis Outline	2
2 Related Work	4
2.1 Continual Learning in Remote Sensing	4
2.2 Continual Learning with Pre-trained Models	5
2.3 Model Merging	6
3 Merging Strategies for Continual Parameter-Efficient Adaptation	8
3.1 Continual Learning	8
3.2 Low-Rank Adaptation	9
3.3 Merging LoRAs for Continual Learning	10
3.3.1 Weight Averaging	11
3.3.2 TIES-Merging	12
3.3.3 Learnable Concatenation (CAT)	13
4 Dataset and Experimental Setup	15
4.1 BigEarthNet v2.0 Dataset Overview	15
4.2 Model Architecture and LoRA Integration	17
4.3 Experimental Setup	19
4.4 Evaluation Metrics	19
4.4.1 Average mAP	21
4.4.2 Average Incremental mAP	21
4.4.3 Forgetting	22
5 Experiments and Results	23
5.1 Baseline Performance	23
5.2 Comparison of Merging Strategies	25
5.3 Controlling Parameter Growth in CAT	27
5.4 Effect of LoRA Rank	28

5.5 Effect of TIES-Merging Keep-Values	30
6 Conclusion and Future Work	32
Bibliography	34

List of Acronyms

AImAP	Average Incremental mean Average Precision
AmAP	Average mean Average Precision
AP	Average Precision
CAT	Learnable Concatenation
CIL	Class Incremental Learning
CLC	CORINE Land Cover
C-LoRA	Continual Low-Rank Adaptation
CL	Continual Learning
DIL	Domain Incremental Learning
EO	Earth Observation
FM	Foundation Models
FN	False Negatives
FP	False Positives
LAE	Unified Continual Learning Framework with General Parameter-Efficient Tuning
LinCAT	Linear Learnable Concatenation
LoRA	Low-Rank Adaptation
mAP	mean Average Precision
MIM	Masked Image Modeling
MLP	Multilayer Perceptron
MSI	Multispectral Imager Instrument
PEFT	Parameter-Efficient Fine-Tuning
PPCA	Probabilistic Principal Component Analysis
PR	Precision-Recall
RS	Remote Sensing
SAR	Synthetic Aperture Radar
SCN	Asymmetric Collaborative Network
TP	True Positives
ViT	Vision Transformer

List of Figures

3.1	The figure illustrates how a low-rank update $\Delta W = BA$, is applied in parallel to the frozen weight matrix W_0 . Adapted from Hu et al. [9].	10
4.1	Two example Sentinel-2 image patches from BigEarthNet v2.0 dataset with the associated multi-labels [33]. Each Patch has a resolution of 120×120 pixels. .	15
4.2	This figure illustrates the different components of the CopernicusFM’s architecture [34].	17
4.3	Example Precision-Recall curves for all 19 classes of the BigEarthNet v2.0 subset used in this thesis. The area under each curve is the corresponding AP value, provided in the Legend. For this example the final upper baseline adapter is used.	20
5.1	This plot serves as a visualization of the baselines performance. For each step t (x-axis), the plot shows the AmAP achieved after training on task T_t (y-axis). .	24
5.2	This plot visualizes the comparison of merging strategies. For each step t (x-axis), the plot shows the AmAP achieved after training on task T_t (y-axis) for each of the merging strategies.	26
5.3	A comparison of merging strategies including LinCAT. For each step t (x-axis), the plot shows the AmAP achieved after training on task T_t (y-axis).	28
5.4	Illustration for the impact of the LoRA rank hyperparameter. The x-axis displays the LoRA rank. The y-axis displays the corresponding metric score for each of the merging methods, evaluated over 3 runs.	29
5.5	A comparison of multiple TIES keep-values, k . For each step t (x-axis), the plot shows the AmAP achieved after training on task T_t (y-axis). This is done for all the different k -values, seen in the legend.	30

List of Tables

4.1	Sentinel-2 Spectral Bands with their respective descriptions, central wavelengths and spatial resolutions [33].	16
4.2	Land-cover classes based on the CORINE CLC 2018 aggregation, with their total occurrences in BigEarthNet v2.0 [33].	16
5.1	Validation mAP (micro) after training only on Task 1 (Finland), evaluated on each single-country's validation set.	24
5.2	Metrics (mean $\pm$ std over 5 runs) for lower and upper baselines.	25
5.3	Metrics (mean $\pm$ std over 5 runs) for baselines and merging strategies.	26
5.4	Metrics (mean $\pm$ std over 5 runs) for comparing LinCAT against the other previously established methods.	27

1 Introduction

1.1 Motivation and Problem Statement

Recent years have shown a rapid increase in the performance of large-scale machine learning models. Traditional deep learning algorithms, tend to be built on a static dataset, where all of the training data is available before the training. However, this approach is often unfeasible in applications such as remote sensing (RS), where there is a continuous stream of newly generated data from satellites or other RS sensors. In such environments, data is generated over time and under changing conditions, such as varying atmospheric circumstances, seasonal shifts in vegetation, gradual land-use modifications, or abrupt events like floods and wildfires. Adapting to these constant changes by retraining models from scratch whenever new information becomes available is often impractical due to high computational costs and time requirements. While access to past data is often limited in other domains by storage or privacy regulations [1], this is typically less of a constraint in RS, where long-term data collection is common and privacy concerns are minimal [2].

A major challenge posed by these dynamic conditions is catastrophic forgetting - a critical problem observed in neural networks when being sequentially trained on new data. It is defined as the overwriting of previously acquired knowledge, which manifests as a significant decline in performance when evaluated on older data [3]. To address this problem continual learning (CL), also referred to as incremental learning or lifelong learning, has emerged - a paradigm that enables models to learn from a stream of data while retaining previously acquired knowledge, without the need of access to old data. The two core objectives of CL are:

1. **Knowledge Acquisition:** To continuously absorb information as the data evolves and changes, enabling the incremental development of new skills.
2. **Forgetting Mitigation:** To overcome the catastrophic forgetting of former knowledge when learning new information [4].

Therefore the idea of CL is to provide a framework suitable for dynamic applications, mitigating catastrophic forgetting and ensuring sustainable model performance [4, 5].

Building on these principles, recent research has increasingly focused on how continual learning can be combined with large pre-trained models. In particular, the continual adaptation of Foundation Models (FM) has gained significant attention in computer vision and other domains. These large-scale models are characteristically pretrained on expansive datasets and aim to serve as universal feature extractors for downstream tasks. FMs demonstrate an exceptional ability to handle a variety of complex tasks by extracting meaningful and versatile information [5]. In

1 Introduction

the context of CL, these powerful, generalized representations are hypothesized to provide a more stable foundation, inherently mitigating catastrophic forgetting [6]. Unfortunately, due to the size of modern FMs, performing full-model fine-tuning has become increasingly impractical in terms of both computation and memory requirements. To address this issue, recent approaches are leveraging pretrained FMs, by not updating all model parameters, but instead using parameter-efficient adaptation (PEFT) methods that fine-tune only a small subset of weights [7, 8].

One example of such a method is Low-Rank Adaptation (LoRA), which has gained popularity for its simplicity and computational efficiency during training and inference, while still achieving competitive performance. LoRA is a PEFT method for adapting large pretrained models by introducing a low-rank decomposition into the weight update process [9]. The relevance of this approach for CL lies in the fact that a separate LoRA module can be created for each newly learned task. This has led to the development of merging techniques that fuse multiple task-specific LoRA adapters into a single one. Originally, these merging strategies were developed in the context of combining expert-specific LoRA modules [10].

1.2 Research Objective

Although LoRA has shown promise for efficient fine-tuning of FMs in static settings, its effectiveness under CL conditions - particularly in the context of RS - has not yet been thoroughly explored. To address this shortcoming, this thesis will investigate the suitability of using LoRA to adapt large FMs, for CL in RS applications. One promising way that might enable LoRA for CL is through merging methods, which allow task-specific LoRA modules to be integrated into a single, compact representation without retraining from scratch. Specifically, three merging strategies - naive Weight Averaging, TIES-merging and Learnable Concatenation (CAT) - are investigated.

The objective is to systematically assess how these techniques influence the trade-off between plasticity (the ability to learn new tasks) and stability (the retention of prior knowledge) across sequential learning stages. In particular, it is examined how fundamental differences between training-based approaches like CAT and training-free methods such as TIES Merging and naive Weight Averaging impact CL performance and scalability across sequential tasks.

1.3 Thesis Outline

This thesis is structured into six main chapters:

- Chapter 2 provides an overview of related work. It first reviews existing studies on CL in RS, followed by an overview of CL with pretrained models and finally some works regarding model merging.
- Chapter 3 presents the methodological groundwork: a formal CL formulation together with the establishments of the used baselines, and the three adapter-merging strategies investigated here – Weight Averaging, TIES-Merging, and CAT.

- Chapter 4 introduces the dataset and experimental setup, the model’s architecture, as well as the used metrics for CL performance evaluation.
- Chapter 5 presents the empirical results in five steps: baseline behavior, a head-to-head comparison of merging strategies, a scalable CAT variant that controls parameter growth (LinCAT), an analysis of the impact of the LoRA rank hyperparameter, and a study of TIES keep-values.
- Chapter 6 concludes the thesis with a summary of key findings and provides suggestions for future research directions.

2 Related Work

This chapter provides an overview of the existing literature relevant to this thesis. It begins by reviewing the landscape of CL within the specific domain of RS, then explores the broader context of CL with large pre-trained models, and finally examines some developments in the field of model merging.

2.1 Continual Learning in Remote Sensing

The rapid increase in RS data collected on a daily basis provides the scale of information where deep learning models excel, making deep learning one of the major fields of study in modern RS applications. Yet, as data distributions shift over time, models trained on fixed datasets often fail to retain past knowledge when exposed to new information. To address this limitation, recent research has explored CL approaches aimed at mitigating catastrophic forgetting and improving the long-term adaptability of deep neural networks in RS environments [1, 4, 5, 7, 8, 11, 12, 13, 14, 15]. Marvellous et al. [15] and Lee et al. [2] organize CL into three canonical categories: replay-based, regularization-based, and architectural-based methods. However, it is important to note that these categories are not mutually exclusive, as many state-of-the-art approaches are hybrids that combine elements from these foundational strategies.

Replay-based methods reintroduce samples from previous tasks while learning new ones, therefore aiming to mitigate catastrophic forgetting. While this is commonly done using a small memory buffer that stores selected real samples from past tasks, an alternative approach uses generative models to create synthetic samples that mimic previous data distributions. During training on new tasks, these old, or synthetic, samples are mixed with the new data, allowing the model to maintain knowledge of earlier distributions. Lee et al. [2] propose a performance-aware submodular sample selection strategy that keeps the most informative and diverse samples. Another replay-based approach was published by Bhat et al. [16], which combines curriculum learning with an informative subset selection strategy within a replay-based framework, showing substantial improvements in stability-plasticity trade-offs on benchmark remote sensing datasets.

Regularization-based methods aim to prevent catastrophic forgetting by adding constraints to the loss function during training. When a neural network learns new tasks, it tends to overwrite parameters that were important for previous tasks. Regularization-based methods penalize such harmful updates by encouraging the model to keep important weights close to their old values. One of the first times this was used in the field of RS was in [17], in which the authors approach integrates Elastic Weight Consolidation with Barlow Twins, leveraging redundancy reduction as an auxiliary constraint to preserve feature representations across tasks [2, 17]. Another noteworthy example for a regularization-based method can be seen in [18]. The paper

proposes a dynamic equilibrium constraint that is added to the loss function. This constraint regularizes the influence of old and new data during training, aiming to create a more balanced learning process.

Architecture-based methods tackle catastrophic forgetting by modifying the model structure itself rather than relying on replay or loss regularization. They allocate separate model components such as modules, layers, neurons or parameters for each new task, often freezing or gating old ones to preserve prior knowledge [2, 15]. Peng et al. [19] propose Asymmetric Collaborative Network (SCN) for class-incremental RS scene classification. SCN uses two structurally distinct subnetworks: a Pres-Net to preserve long-term, task-general knowledge and a Trans-Net to learn task-specific features for the current increment. During each task, historical features are recalled from Pres-Net to Trans-Net via a recall filter. After learning, knowledge is distilled back from Trans-Net to Pres-Net with a triple-distillation loss. Trans-Net is then discarded, so inference uses only the updated Pres-Net.

Out of these three, architecture-based methods remain the most unexplored in the field of RS - especially for multispectral data - indicating a gap in existing research. One reason for that may be, that these methods tend to increase model complexity as tasks accumulate over time [2]. This thesis aims to contribute to addressing this gap by investigating whether the merging of LoRA-based modules can function as a potential architecture approach to CL in RS using multispectral Sentinel-2 data.

2.2 Continual Learning with Pre-trained Models

Pretraining changes the starting point of CL: instead of learning representations from scratch, a model begins with strong, well-generalized features in the latent space. Since pre-trained models are often large and computationally expensive to retrain, most approaches avoid full fine-tuning and instead adopt more efficient adaptation strategies. Common methods either keep the backbone frozen entirely, or adapt it with small, targeted updates. By starting from a pre-trained model, that already captures broad representations, it becomes less sensitive to distribution shifts and requires fewer updates to integrate new knowledge. This stabilizes the learning process and helps mitigate catastrophic forgetting across sequential tasks [5, 6, 7, 8].

In [5], Boum et al. investigate Domain Incremental Learning (DIL) in the context of RS. The paper proposes the combination of a frozen FM backbone with a generative Probabilistic Principal Component Analysis (PPCA) classifier to reduce partial forgetting when new domains are introduced. Instead of retraining models, the method keeps the pre-trained feature extractor fixed and incrementally updates the PPCA classifier using the means and covariances from new domains. The results of the study highlight that FMs pre-trained on large, general-purpose vision datasets outperform RS-specific models trained on more specialized, smaller scale ones. However, the study did not consider any FMs pre-trained on multispectral or hyperspectral data.

Janson et al. [6] raise the question, whether complex CL mechanisms are truly necessary when large pre-trained models are used. The paper proposes a very simple baseline based on a frozen pre-trained Vision Transformer (ViT): ViT-B/16. For Class Incremental Learning (CIL) the method first extracts image features using the frozen backbone, then computes a class mean vector for each class, and classifies new samples by nearest-mean distance in the feature space.

2 Related Work

For DIL the same procedure is applied, but the class means are continually updated as new domains with the same classes are observed, allowing the model to gradually adapt its class prototypes to domain shifts without any parameter updates or retraining.

With the increasing popularity of large pre-trained models - especially foundation models - the need for efficient adaptation methods has grown substantially. To address this, PEFT methods have emerged, which adapt pre-trained models by introducing a small number of additional trainable parameters, usually in the form of adapters, prompts or low-rank matrices, while keeping the original model's weights frozen. These techniques significantly reduce the cost of adaptation and have become a key paradigm for leveraging pre-trained models in continual and multi-task learning settings [7, 8].

Gao et al. [7] present a unified CL framework (LAE) to CL that leverages multiple PEFT methods such as Adapters, LoRA and Prefix/Prompt tuning. Unlike prior prompt-based methods limited to Transformers, LAE is architecture-agnostic and applicable to both Transformers and ConvNets. The framework introduces three core components: an online PEFT module for adapting new tasks with calibrated learning speed, an offline module that accumulates past knowledge through momentum updates, and an ensemble strategy that combines both experts during inference. LAE enables memory-free CL, achieving state-of-the-art results on common CL benchmark datasets.

Zhang et al. [8] introduce Continual Low-Rank Adaptation (C-LoRA), a continual version of the prominent PEFT method LoRA. Unlike existing LoRA frameworks, which utilize a growing set of task-specific adapters, C-LoRA uses a single low-rank adapter controlled by a learnable routing matrix. The matrix is split into a frozen part and a trainable increment. An orthogonality regularizer and a stop-gradient constrain updates, so new tasks reuse subspaces without overwriting old ones. Integrated into ViT Multilayer Perceptron (MLP) blocks, this yields parameter-efficient CIL without per-task adapter selection. On several established benchmark datasets and across various sequential learning sessions, C-LoRA reports higher Last-Acc and Inc-Acc than strong regularization/prompt/adapters baselines, while using fewer additional parameters.

Taken together, these studies outline a clear progression in how pre-trained- and FMs are leveraged for CL. While Boum et al. demonstrates the general benefit of frozen foundation models in DIL RS settings, Janson et al. show that even minimal adaptation can yield competitive results when pre-trained representations are sufficiently strong. Building upon this, Gao et al. and Zhang et al. investigate how PEFT methods can enable CL by allowing models to adapt efficiently while keeping the backbone frozen. However, in the context of multispectral RS, the question, if the merging of LoRA adapters can be used as a viable approach for CL, remains largely unexplored.

2.3 Model Merging

Model merging has become an increasingly relevant research direction in recent years, aiming to combine the knowledge of multiple models into a single, unified system. Rather than training a single model from scratch on the combined data from all tasks, merging techniques operate directly on existing model parameters or modular components, allowing for efficient knowledge

integration across tasks or domains [10, 20, 21].

Ruan et al. [20] provide a comprehensive overview, review and taxonomy of model merging methods.

Although in this thesis Weight Averaging as a merging method was not directly based on any single prior publication, related works including [22] by Wortsman et al. and [23] by Choshen et al. address similar ideas of merging models through Weight Averaging.

While Weight Averaging represents the simplest form of model merging, its limitations, among them parameter interference, led to the proposal of more advanced methods: TIES-Merging [21] extends Weight Averaging by trimming redundant weights and aligning parameter signs before averaging. Consensus Merging [24] uses TALL-masks to analyze the merged model and filter out *selfish* or *catastrophic* weights. Another strategy re-frames the problem in the frequency domain, where FREE-Merging [25] uses high-pass filters to remove interfering low-frequency signals from task vectors before combining them. Separate from the interference-focused techniques, a LoRA-specific method, LoRA-Soups [10] uses learnable concatenation (CAT), where trainable coefficients are optimized to combine multiple skills.

Within the context of merging LoRA adapters for CL, this thesis evaluates three of these methods to determine whether increasing merging complexity, translates into measurable performance gains. The chosen approaches progress from simple averaging (Weight Averaging), to more structured interference-aware merging (TIES), and finally to learnable merging (CAT).

The specific details of these three approaches are discussed in Chapter 3.

3 Merging Strategies for Continual Parameter-Efficient Adaptation

This chapter establishes the theoretical foundation for the methods investigated in this thesis. It starts with a formal definition of CL and its primary paradigms. Subsequently, it provides a detailed explanation of LoRA and describes the training and merging process, including the three merging strategies central to this work: Weight Averaging, TIES-Merging and Learnable Concatenation.

3.1 Continual Learning

Problem Formulation. Let a learning model be represented by a function f_θ with parameters θ . In a CL scenario, the model is exposed to a sequence of tasks, $T = \{T_1, T_2, \dots, T_N\}$. Each task T_i is associated with a data distribution D_i over a feature space X and a task-specific label space Y_i . Therefore, a data point (x, y) is drawn from the distribution D_i , where $x \in X$ is the input image and $y \subseteq Y_i$ is the corresponding set of labels.

The objective of a CL algorithm is to learn the parameters θ that performs well on all tasks seen so far. At any given step n , having observed tasks $T_1, \dots, T_n$, the goal is to find the optimal parameters θ_n^* that minimize the average loss, where $\mathcal{L}$ is a predefined loss function [26, 27, 28, 29]:

$$\theta_n^* = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{(x,y) \sim D_i} [\mathcal{L}(f_\theta(x), y)] \quad (3.1)$$

The main focus of this thesis lies on DIL, a CL scenario where the model must learn to perform the same learning task under changing input data distributions, referred to as "domains". The core assumption is that the set of classes, and thus the learning task itself, remains constant throughout the learning process. Building on the notation above, the defining characteristic of the DIL setting is that the label space is identical for all tasks:

$$\forall i, j \in \{1, \dots, N\} : Y_i = Y_j \quad (3.2)$$

The increment between tasks occurs exclusively in the input domain. This means the marginal probability distribution of the input data, $P_i(X)$, differs between tasks (i.e., $P_i(X) \neq P_j(X)$ for $i \neq j$), while the conditional probability of the labels given the input, $P_i(Y|X)$, also shifts to reflect the new domain [26, 30].

To quantify the impact of sequential learning in this DIL setting and provide a consistent reference for all results, two baselines are introduced: a lower baseline that performs naive sequential training and an upper baseline that retrains from scratch on the accumulated data at each step.

The **lower baseline** simulates a direct sequential training process without any mechanism to preserve past knowledge. In this setup, the model is trained on the first task, and for each subsequent task, training continues from the final weights of the preceding task. This approach directly exposes the model to catastrophic forgetting, where the optimization process for a new task overwrites the parameters crucial for previous tasks. It serves as a benchmark because it quantifies the severity of the forgetting problem in the absence of any mitigation strategies. Any successful CL method must outperform this baseline.

Formally if we denote the trainable parameters – located in the LoRA adapter and classification head – after task t as θ_t and the dataset as D_t , the lower baseline training process can be described as:

$$\theta_t^{LB} = \text{Train}(\theta_{t-1}^{LB}, D_t), \quad \text{for } t > 1 \quad (3.3)$$

The **upper baseline**, in contrast, represents the best-case performance achievable with the given model and data, assuming no constraints on data access or computation. For each step in the task sequence, this baseline is established by retraining the model from scratch on the cumulative dataset of all tasks encountered up to that point. This approach avoids forgetting entirely by re-exposing the model to all past data during each training cycle. While computationally expensive and therefore often impractical in real-world scenarios, it provides a theoretical performance ceiling. The gap between each CL strategy and this upper baseline indicates the performance cost incurred by learning sequentially.

This process can be formally defined as:

$$\theta_t^{UB} = \text{Train}(\theta_0, D_1 \cup \dots \cup D_t) \quad (3.4)$$

3.2 Low-Rank Adaptation

LoRA is a parameter-efficient adaptation method designed to fine-tune large, pre-trained models for specific downstream tasks. It is most commonly applied in Transformer architectures, on the linear layers within the self attention blocks. The method is based on the hypothesis that the change in weights during model adaptation exhibits a low intrinsic rank, a claim supported by other works such as Aghajanyan et al. [31]. Therefore, instead of updating all model parameters, LoRA freezes the original pre-trained weights and injects trainable, low-rank matrices into the existing architecture. Let W_0 be a given weight matrix within a Transformer layer with $W_0 \in \mathbb{R}^{d \times k}$. Instead of learning the full-rank weight update ΔW directly, it is approximated by the product of two smaller matrices A and B [9]:

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k} \quad (3.5)$$

Here, the rank r is a hyperparameter that is significantly smaller than the original dimensions of the weight matrix, $r \ll \min(d, k)$. While W_0 remains frozen throughout the training process, only the parameters of matrices A and B are optimized. The modified forward pass h of the layer is described as [9]:

$$h = W_0 x + \Delta W x = W_0 x + B A x \quad (3.6)$$

3 Merging Strategies for Continual Parameter-Efficient Adaptation

To ensure a stable training process and to preserve the knowledge stored in the pre-trained model at the beginning of the adaptation, the matrix A is initialized with values from a Gaussian distribution, while the matrix B is initialized with zeros. As a result the update term $\Delta W = BA$ is zero at the start of training, making the adapted model initially identical to the pre-trained one (see Figure 3.1). Furthermore the adaptation is modulated by the hyperparameter α , serving as a scaling factor. The complete formulation of the forward pass incorporates this scaling, which works in tandem with the rank r . The update is applied as [9]:

$$h = W_0 x + \frac{\alpha}{r} B A x \quad (3.7)$$

The factor $\frac{\alpha}{r}$ serves as a normalization term. It ensures that the overall magnitude of the adaptation remains consistent, regardless of the chosen rank r . Without this scaling, increasing the rank would also increase the effective magnitude of the update, potentially destabilizing the training and requiring retuning of other hyperparameters like the learning rate [9].

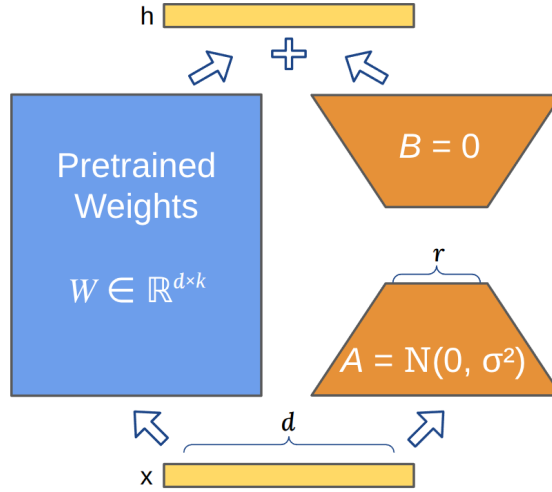


Figure 3.1: The figure illustrates how a low-rank update $\Delta W = BA$, is applied in parallel to the frozen weight matrix W_0 . Adapted from Hu et al. [9].

3.3 Merging LoRAs for Continual Learning

A crucial aspect of the experimental design is that the parameters of the pre-trained FM backbone remain frozen throughout all training stages. For any given task t , learning is restricted to a set of trainable parameters, θ_t , which exclusively consists of the LoRA matrices and the task-specific classification head, $\theta_t = \{\theta_t^{LoRA}, \theta_t^{Head}\}$. The merging procedures are therefore applied only to these trainable components.

The process begins with an initialization stage on the first task, T_1 , where the trainable parameters θ_1^* are learned on top of the frozen FM backbone. This initial model serves as the foundation

3.3 Merging LoRAs for Continual Learning

for the subsequent incremental cycle. For each following task t , a two-stage incremental learning cycle is executed. This cycle begins with a continual training phase, where the model's trainable parameters from the previous consolidated state, θ_{t-1}^* , are used to initialize a new training run on the current task's data, $\mathcal{D}_t$. This adaptation yields a new set of task-specialized parameters, θ'_t .

$$\theta'_t = \text{Train}(\theta_{t-1}^*, \mathcal{D}_t) \quad (3.8)$$

Following this, a merging stage consolidates the newly acquired knowledge. To create a unified model that integrates information from all preceding tasks, the parameters of the previous consolidated model, θ_{t-1}^* , are merged with the most recently trained task-specialized parameters, θ'_t , which encapsulates the knowledge from task 1 through t .

$$\theta_t^* = \text{Merge}(\theta_{t-1}^*, \theta'_t) \quad (3.9)$$

The specific implementation of the $\text{Merge}(\cdot)$ function is contingent on the evaluated strategy. For Weight Averaging and TIES-Merging, the algorithmic logic is applied symmetrically to both the LoRA adapters and the classification heads, merging each component independently. In the case of CAT, a hybrid approach is necessary as the source literature [10] does not specify a merging procedure for classification heads. Consequently, the classification heads are first merged using standard weight averaging. Subsequently, the LoRA adapters are combined using the CAT methodology, which uses the learnable scaling coefficients λ , introduced in chapter 3.3.3. This is followed by a brief joint optimization phase of a few epochs, where both the coefficients λ and the weights of the newly averaged classification head are fine-tuned on a small, balanced subset composed 5% of the training data from all tasks encountered thus far.

3.3.1 Weight Averaging

Let $W_1, \dots, W_N \in \mathbb{R}^{d \times k}$, be $N \geq 2$ versions of the same layer coming from different models of identical architecture. Weight averaging, in the context of this thesis, defines the merged weight $\bar{W}$, as the elementwise arithmetic mean:

$$\bar{W} = \frac{1}{N} \sum_{i=1}^N W_i \quad (3.10)$$

For a LoRA-parameterized layer, let the frozen base be $W_0 \in \mathbb{R}^{d \times k}$ and for each task-specific adapter i let the low-rank update be

$$\Delta W_i = B_i A_i, \quad B_i \in \mathbb{R}^{d \times r}, \quad A_i \in \mathbb{R}^{r \times k} \quad (3.11)$$

with a common rank r and compatible dimensions across all adapters. Weight averaging as described above would be applied as:

$$\bar{A} = \frac{1}{N} \sum_{i=1}^N A_i, \quad \bar{B} = \frac{1}{N} \sum_{i=1}^N B_i, \quad \Delta W_{\text{merge}} = \bar{B} \bar{A} \quad (3.12)$$

3 Merging Strategies for Continual Parameter-Efficient Adaptation

A drawback of this method is created by the generation of cross terms within ΔW_{merged} . It becomes clear when expanding the product $\bar{B}\bar{A}$:

$$\bar{B}\bar{A} = \left(\frac{1}{N} \sum_{i=1}^N B_i\right) \left(\frac{1}{N} \sum_{j=1}^N A_j\right) = \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N B_i A_j = \frac{1}{N^2} \left(\sum_{i=1}^N B_i A_i + \sum_{\substack{i,j=1 \\ i \neq j}}^N B_i A_j \right) \quad (3.13)$$

The summands $B_i A_j$ with $i \neq j$ are the cross terms. One hypothesis regarding their performance impact is that they couple A and B matrices from different adapters and therefore do not align with a single task-consistent update direction, which can introduce interference. Empirical evidence supporting this behavior has been provided by Tang et al. [32].

3.3.2 TIES-Merging

While the weight averaging, as described in the previous chapter, provides a direct method for merging, its performance can be impacted by parameter interference. To address this, Yadav et al. [21] introduced TIES-Merging, a methodology that refines the averaging process by systematically resolving interference from redundant parameters and sign disagreements. It is built upon the concept of the task vector. A task vector $\tau_t \in \mathbb{R}^d$ represents the displacement in the d -dimensional parameter space that a model undergoes when fine-tuned for a specific task t . As formalized in [21], it is defined as the difference between the fine-tuned model's parameters and the original pre-trained parameters:

$$\tau_t = \theta_t - \theta_{init} \quad (3.14)$$

Here, $\theta_{init} \in \mathbb{R}^d$ represents the initial weights of the pre-trained model, and $\theta_t \in \mathbb{R}^d$ represents the weights after fine-tuning on task t . By calculating this difference, the task vector effectively isolates the specific changes learned during adaptation, separating them from the shared knowledge of the base model. Within this thesis, this concept is applied directly to the LoRA adapters themselves. Each adapter represents a complete, self-contained modification. Because all adapters are trained to augment the same frozen backbone model $W_0 \in \mathbb{R}^{d \times k}$, they operate relative to this common origin. The adapter's weights, therefore, capture the total task-specific displacement from the backbone's frozen state. For this reason, each LoRA adapter can be seen as a task vector in its entirety. The TIES-Merging process is therefore applied independently to the set of all A matrices $\{A_1, \dots, A_N\}$ and the set of all B matrices $\{B_1, \dots, B_N\}$. This process is executed in three main stages – Trim, Elect Sign, and Merge – which are now described in detail.

Trim. The first step, Trim, addresses the potential issue where certain parameters in the weight matrices undergo minor changes that have little impact on performance but potentially obscure more significant updates during averaging. This step identifies and retains only the top- $k\%$ weights with the highest absolute magnitude within each A_t and B_t matrix. The remaining $(100 - k)\%$ of weights are reset to zero. This creates trimmed matrices $\hat{A}_t$ and $\hat{B}_t$, that contain only the most influential parameters for each task. Formally, for each matrix $M \in \{A_t, B_t\}$:

$$\hat{M}_{i,j} = \begin{cases} M_{i,j} & \text{if } |M_{i,j}| \text{ is in the top-}k\% \text{ of } |M| \\ 0 & \text{otherwise} \end{cases} \quad (3.15)$$

3.3 Merging LoRAs for Continual Learning

This pruning step isolates the most critical weights, preventing interference from the low-magnitude, redundant parameters.

Elect Sign. The second step, Elect Sign, resolves directional conflicts where different adapters propose updates of opposing signs for the same weight parameter. Naively averaging these values, like in the method above, would cause them to cancel each other out, diminishing their intended effect. To establish a consensus direction, an aggregated sign matrix is computed for each set of trimmed LoRA matrices. This is achieved by summing the trimmed matrices and taking the sign of the result, therefore selecting the sign of greater total magnitude for each entry. This produces two elected sign matrices, Γ_A and Γ_B :

$$\Gamma_A = \text{sgn}\left(\sum_{t=1}^N \hat{A}_t\right) \quad \text{and} \quad \Gamma_B = \text{sgn}\left(\sum_{t=1}^N \hat{B}_t\right) \quad (3.16)$$

These matrices, Γ_A and Γ_B , define the consensus direction of change for every individual weight in the final merged adapter.

Merge. The final step, Merge, computes the average using only the weights that align with the elected sign direction. This prevents the just mentioned dilution of parameter magnitudes caused by averaging values with opposing signs. For each parameter position (i, j) in the A and B matrices the final merged weights $A_{m,i,j}$ and $B_{m,i,j}$ are calculated by averaging only the non-zero values from the trimmed matrices whose sign matches the corresponding sign in Γ_A and Γ_B .

Let $S_{A,i,j} = \{t \mid \text{sgn}(\hat{A}_{t,i,j}) = \Gamma_{A,i,j}\}$ be the set of adapters whose sign for parameter (i, j) in matrix A agrees with the consensus. The final merged matrices, A_m and B_m , are:

$$A_{m,i,j} = \frac{1}{|S_{A,i,j}|} \sum_{t \in S_{A,i,j}} \hat{A}_{t,i,j} \quad \text{and} \quad B_{m,i,j} = \frac{1}{|S_{B,i,j}|} \sum_{t \in S_{B,i,j}} \hat{B}_{t,i,j} \quad (3.17)$$

The resulting pair of matrices $\{A_m, B_m\}$, forms the new, merged LoRA adapter.

3.3.3 Learnable Concatenation (CAT)

In contrast to the training-free methods described previously, which average existing parameters within a fixed-rank architecture, CAT offers a different paradigm for merging. CAT is adapted for this thesis and is based on the work "LoRA Soups" by Prabhakar et al. [10]. The core idea is to find an optimal, learned weighting for the skills contributed by each of N task task-specific adapters, rather than relying on a static merging rule. The merging operation is applied independently to each targeted layer within the frozen backbone model. For a given target layer, the low-rank matrices from N adapters, $\{A_i, B_i\}_{i=1}^N$, are concatenated. The A matrices are concatenated vertically and the B matrices are concatenated horizontally, creating a new, larger matrix pair $\{A_m, B_m\}$ for that specific layer. The rank of this new pair is the sum of the individual

3 Merging Strategies for Continual Parameter-Efficient Adaptation

adapter ranks, $r_m = \sum_{i=1}^N r_i$. Formally, for each layer, the merged matrices are constructed as:

$$A_m = \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_N \end{bmatrix}, \quad B_m = \begin{bmatrix} B_1 & B_2 & \cdots & B_N \end{bmatrix} \quad (3.18)$$

While a simple concatenation, would treat the contributions from all adapters equally, the main idea of CAT lies in the introduction of N trainable scaling coefficients, $\{\lambda_1, \dots, \lambda_N\}$, that modulate the influence of each original adapter. This learning process is also conducted layerwise, meaning a distinct set of λ weights is optimized for each layer, targeted by LoRA. The effect of these learned weights is to create a final weight update $\Delta W_{m,l}$ for each layer l , that is a weighted linear combination of the updates from the original adapters:

$$\Delta W_{m,l} = \sum_{i=1}^N \lambda_{i,l} B_{i,l} A_{i,l} \quad (3.19)$$

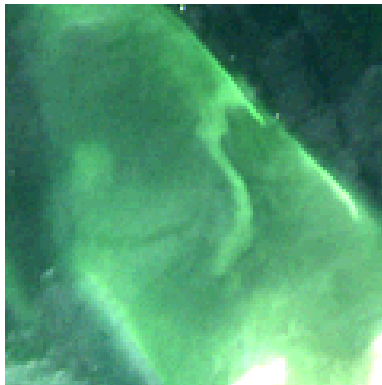
Once the optimal values for each $\lambda_{i,l}$ are learned, they are permanently fused into their corresponding concatenated $B_{m,l}$ matrix by scaling the columns of $B_{m,l}$ associated with each original adapter i . To preserve the stability of the training process, the LoRA scaling hyperparameter α is also being adjusted. The original LoRA formulation maintains a consistent update magnitude by normalizing the update with the term $\frac{\alpha}{r}$. However, as the concatenation increases the rank r , without a corresponding change in α , the effective update magnitude would be reduced by a factor of N . To counteract this and maintain the original $\frac{\alpha}{r}$ ratio, the α parameter is rescaled proportionally to the increase in rank. It is noteworthy that the work by Prabhakar et al. [10] focuses primarily on skill composition - combining distinct, pre-trained abilities to solve a single, more complex target task. This thesis, in contrast, investigates whether the CAT merging strategy can be repurposed as a mechanism for CL. Given that the core challenge of CL is the mitigation of catastrophic forgetting rather than skill composition, it is an open question whether this merging strategy remains effective in such a different operational environment.

4 Dataset and Experimental Setup

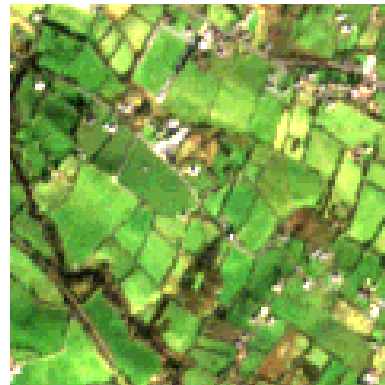
This chapter details the experimental setup used to evaluate the merging strategies. It introduces the BigEarthNet-V2 dataset [33], describes the model architecture and LoRA integration, outlines the experimental setup, and defines the metrics used to performance evaluation.

4.1 BigEarthNet v2.0 Dataset Overview

BigEarthNet v2.0 [33] is a large scale, multi-modal benchmark dataset designed for deep learning applications in RS. It consists of 549,488 pairs of image patches (see Figure 4.1) from two Sentinel satellites, Sentinel-1 and Sentinel-2. The images were captured between June 2017 and May 2018 across 10 European Countries: Austria, Belgium, Finland, Ireland, Kosovo, Lithuania, Luxembourg, Portugal, Serbia and Switzerland.



(a) Marine Waters



(b) Pastures, Urban fabric

Figure 4.1: Two example Sentinel-2 image patches from BigEarthNet v2.0 dataset with the associated multi-labels [33]. Each Patch has a resolution of 120×120 pixels.

The multispectral images are sourced from the Sentinel-2 satellite’s Multispectral Imager instrument, which captures data across 13 spectral bands at three different spatial resolutions (see Table 4.1), depending on the band. Each image patch corresponds to a spatial area of $1.2 \text{ km} \times 1.2 \text{ km}$ [33].

4 Dataset and Experimental Setup

Table 4.1: Sentinel-2 Spectral Bands with their respective descriptions, central wavelengths and spatial resolutions [33].

Band	Description	Central Wavelength [nm]	Resolution [m]
B1	Coastal Aerosol	443	60
B2	Blue	490	10
B3	Green	560	10
B4	Red	665	10
B5	Red Edge 1	705	20
B6	Red Edge 2	740	20
B7	Red Edge 3	783	20
B8	NIR	842	10
B8A	Narrow NIR	865	20
B9	Water Vapour	940	60
B11	SWIR 1	1610	20
B12	SWIR 2	2190	20

Table 4.2: Land-cover classes based on the CORINE CLC 2018 aggregation, with their total occurrences in BigEarthNet v2.0 [33].

Land-cover class	Total occurrences
Arable land	188025
Mixed forest	165780
Coniferous forest	154941
Transitional woodland, shrub	141150
Broad-leaved forest	135928
Land principally occupied by agriculture, with significant areas of natural vegetation	122709
Complex cultivation patterns	99598
Pastures	95605
Marine waters	61832
Inland waters	63212
Urban fabric	63758
Agro-forestry areas	33181
Permanent crops	29588
Inland wetlands	20919
Moors, heathland and sclerophyllous vegetation	13894
Natural grassland and sparsely vegetated areas	11882
Industrial or commercial units	11142
Coastal wetlands	1397
Beaches, dunes, sands	1316

In addition to the optical data, each patch is paired with Synthetic Aperture Radar (SAR) data from the Sentinel-1 mission. This provides structural information about the Earth’s surface. The ground truth for BigEarthNet v2.0 is derived from the CORINE Land Cover (CLC) database of 2018. Each image patch can be associated with one or more land cover classes simultaneously [33]. The dataset uses a nomenclature of 19 distinct classes for evaluation and training, as seen in Table 4.2.

4.2 Model Architecture and LoRA Integration

The experimental work in this thesis is build upon the CopernicusFM [34], a versatile Vision Transformer (ViT) architecture designed specifically for Earth Observation (EO). The selection of CopernicusFM is primarily driven by its architectural flexibility, while also demonstrating high performance on diverse EO benchmarks, including BigEarthNet v2.0. A key advantage of the model is its ability to process data from various spectral sensors without requiring manual adjustments to its architecture, allowing for a fast and easy implementation. In order to balance performance with computational feasibility, the ViT-B/16 version of the model is utilized for all experiments [34].

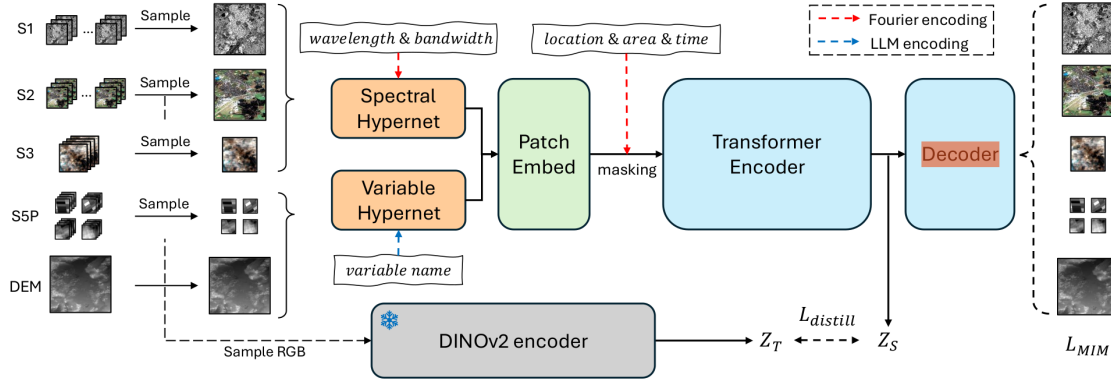


Figure 4.2: This figure illustrates the different components of the CopernicusFM’s architecture [34].

The model’s data processing pipeline, depicted in Figure 4.2, can be split into three main stages: dynamic patch embedding via sensor-aware hypernetworks, metadata integration, and the self-supervised training objective of Masked Image Modeling (MIM) using a Transformer encoder-decoder structure [34].

Beginning with the first stage, a notable aspect of the CopernicusFM architecture is its approach to patch embedding. It diverges from other ViT models by replacing the conventional fixed patch embedding layer with a dynamic mechanism that generates modality-specific patch embeddings on the fly. This is orchestrated by two hypernetworks. The **Spectral Hypernetwork** is responsible for processing data from spectral sensors, such as the multispectral imagery from Sentinel-2. The hypernetwork takes the physical properties of each spectral channel –

4 Dataset and Experimental Setup

its central wavelength and bandwidth – as input. These numerical values are transformed into high-dimensional vector representations using a Fourier encoding scheme. The resulting encoded vectors are then processed by a small network consisting of MLPs and attention layers. This network outputs the specific weights and biases of a 2D convolutional kernel. In essence, the model learns to generate a unique convolutional filter for each input channel based on its spectral characteristics. This dynamic kernel generation is what allows CopernicusFM to be sensor-agnostic, enabling the model to process any modality with a spectral response regardless of spatial and channel dimensions. As it can not naively process non-spectral modalities, CopernicusFM utilizes a second **Variable Hypernetwork**, for the processing of non-spectral data by encoding their semantic variable names [34]. However, since this thesis exclusively utilizes multispectral Sentinale-2 data, this component is not used and is therefore not detailed further.

Within the second stage of the pipeline, the model integrates contextual **metadata** to enrich the patch representations. Specifically, the model is designed to incorporate the latitude and longitude of the image’s center, the spatial area covered by the image patch and the acquisition time of the image capture. To process these inputs, the model employs the same Fourier encoding scheme utilized by the spectral hypernetwork. The resulting metadata vectors are then added directly to the sequence of patch tokens – alongside the standard positional encoding – before the entire sequence is passed to the Transformer encoder [34].

For the third and final stage CopernicusFM is pre-trained using MIM, a self-supervised learning strategy that tasks the model with reconstructing masked portions of an input image. This process leverages an asymmetric encoder-decoder architecture. After the patch embedding and metadata integration stages, 70% of the patch tokens are randomly masked. The remaining sequence of visible patches is then processed by the main Transformer **Encoder**, a standard ViT-Base model consisting of 12 Transformer blocks and 768 hidden dimensions. The encoder’s task is to learn a latent representation of the image content from only the partial input. Subsequently, a separate and more lightweight Transformer **Decoder**, composed of 8 Transformer blocks, undertakes the reconstruction task. This decoder receives both the contextualized feature representations of the visible patches from the encoder, along with the learnable mask tokens that serve as placeholders for the missing patches. By leveraging the context provided by the visible patches, the decoder is trained to predict the original pixel values of the masked patches, forcing the encoder to develop robust and generalizable feature representations in the process [34].

In this work, the LoRA methodology is applied to the linear projection layers for query, key and value (qkv) within the multi-head self-attention mechanism, the output projection layer (proj) that aggregates the outputs from the attention heads, and both linear layers (fc1 and fc2), of the feed-forward network that follows each self-attention module. Collectively, these targeted layers contain the vast majority of the parameters within each Transformer block, making them the primary focus for adaptation. The PEFT configuration for experiments consisted of $r = 8$, $\alpha = 16$, $lr = 2e - 4$, weight decay of 0.01 and a dropout of 0.05. The influence of the LoRA rank is analyzed in further detail in chapter 5.5 Effect of LoRA Rank.

4.3 Experimental Setup

The empirical evaluation of the merging strategies is conducted within a simulated CL environment, constructed from the BigEarthNet v2.0 dataset. To establish this environment, the dataset is partitioned by country, into a sequence of six tasks, $T = \{T_1, T_2, \dots, T_6\}$. The tasks are introduced to the model in a fixed order: Finland (T_1), Portugal (T_2), Serbia (T_3), Lithuania (T_4), Ireland (T_5), and Austria (T_6). While the input data distribution changes with each task-specific dataset, $\mathcal{D}_t$, the learning objective – multi-label classification – remains consistent throughout the entire sequence. Each $\mathcal{D}_t$ is composed of 10,000 samples, which are subdivided into training (7000), validation (1,500), and test (1500) sets. All countries not containing enough samples to fulfill this split, are dropped.

The experimental pipeline is implemented using the PyTorch Lightning framework, with the ConfigILM [35] library managing all data loading and batching operations. Within this framework, a checkpoint strategy is employed to ensure the propagation of optimal model states throughout the CL sequence. While early stopping is utilized to prevent unnecessary computation time, the system specifically saves the model parameters from the epoch that achieved the highest mean Average Precision (mAP) score on the validation set. This "best" state is then used for the subsequent training of the following task.

4.4 Evaluation Metrics

The evaluation framework for the CL experiments is based on the mAP metric as the fundamental performance measure. From this core metric, three indicators of CL capability are derived: Average mAP (AmAP), Average Incremental mAP (AImAP), and Forgetting. A formal definition of mAP and its underlying components is therefore a necessary prerequisite for understanding the subsequent CL metrics.

The foundational concepts for using mAP within evaluation metrics, are precision and recall. Precision is the fraction of true positive predictions among the total number of instances classified as positive by the model [36, 37].

$$Precision = \frac{TP}{TP + FP} \quad (4.1)$$

Here, TP denotes True Positives and FP denotes False Positives. It measures the exactness of a classifier. Recall on the other hand, is the fraction of true positive predictions that the model correctly identifies out of all actual positive instances in the dataset [36, 37].

$$Recall = \frac{TP}{TP + FN} \quad (4.2)$$

where FN denotes False Negatives. It measures the completeness of a classifier. The relationship between precision and recall across different classification thresholds is visualized by the Precision-Recall curve. The area under this curve corresponds to the Average Precision (AP)

4 Dataset and Experimental Setup

for a single class, a scalar value that summarizes its performance. A high-performing model maintains a high precision as recall increases, which results in a PR curve positioned towards the top-right corner of the plot, as illustrated by the example in Figure 4.3. The mAP score is then computed as the arithmetic mean of these individual AP values across all N classes [36, 37].

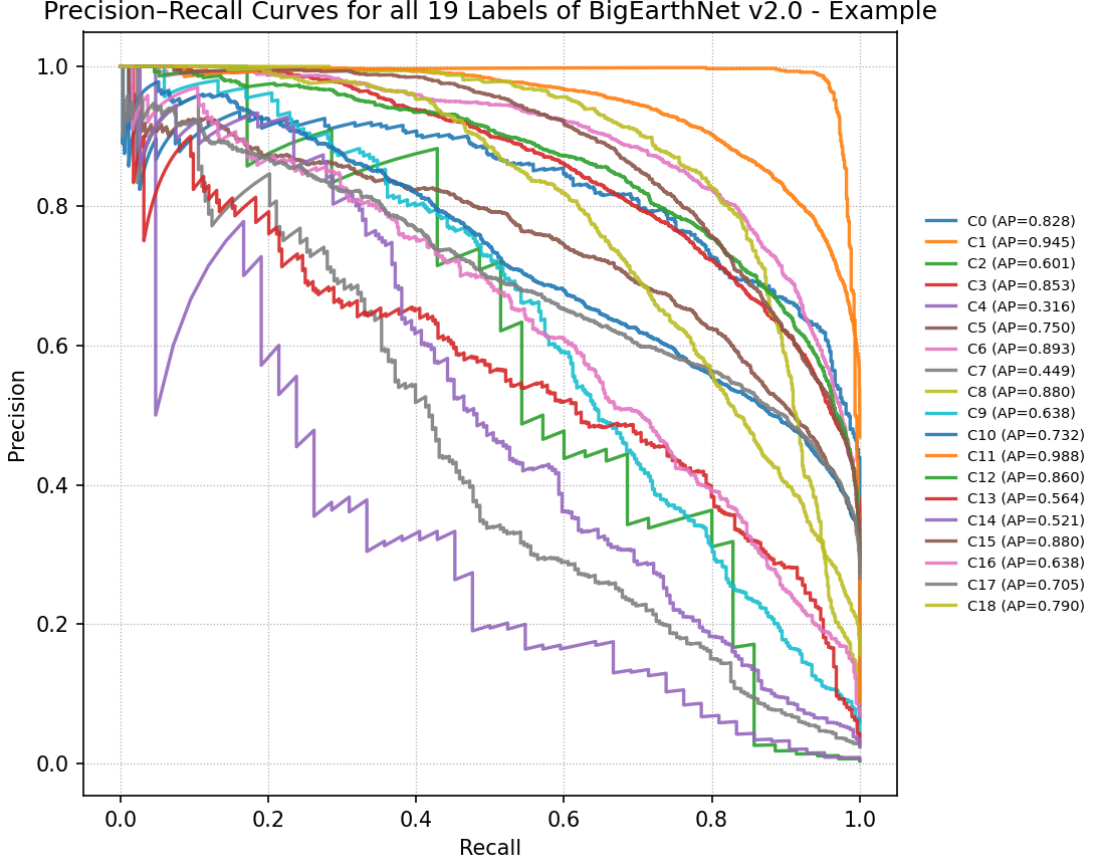


Figure 4.3: Example Precision-Recall curves for all 19 classes of the BigEarthNet v2.0 subset used in this thesis. The area under each curve is the corresponding AP value, provided in the Legend. For this example the final upper baseline adapter is used.

In the context of multi-label classification, mAP is computed using two primary aggregation strategies: macro- and micro-averaging. Macro-averaging computes the AP for each class independently and then calculates the unweighted arithmetic mean of these per-class scores [36, 37].

$$mAP_{macro} = \frac{1}{N} \sum_{k=1}^N AP_k \quad (4.3)$$

This method assigns equal weight to each class, regardless of its instance frequency. As a result, poor performance on infrequent classes is heavily penalized, as their low AP scores contribute

to the final average just as much as the scores from the more prevalent classes [38].

In contrast micro-averaging aggregates the individual TP , FP , and FN counts across all classes before calculating the metric. The micro-averaged precision and recall are thus defined as:

$$Precision_{micro} = \frac{\sum_{k=1}^N TP_k}{\sum_{k=1}^N TP_k + \sum_{k=1}^N FP_k} \quad (4.4)$$

$$Recall_{micro} = \frac{\sum_{k=1}^N TP_k}{\sum_{k=1}^N TP_k + \sum_{k=1}^N FN_k} \quad (4.5)$$

The mAP-Micro is then the area under the PR curve constructed from these globally aggregated values. While mAP-Macro is sensitive to performance on rare classes, mAP-Micro provides a measure of aggregate performance by weighting every individual label prediction equally. The resulting score therefore reflects overall performance from a per-prediction perspective, where the model's ability to correctly classify a prevalent label holds the same importance as its ability to correctly classify a rare one [37, 38].

Given the significant class imbalance within the dataset (see Figure 4.2), the mAP-Macro score is highly sensitive to the performance of those rare classes. mAP-Micro is therefore used as the primary evaluation metric throughout this thesis unless otherwise specified. In the following sections the used metrics are formally described.

Let $R \in \mathbb{R}^{T \times T}$ be a metric matrix, where T is the total number of sequential tasks. Any element $R_{i,j}$ in this matrix represents the mAP achieved on task j after training on tasks 1 through i . Using R , the three CL evaluation metrics mentioned above can be defined as follows.

4.4.1 Average mAP

AmAP provides a snapshot of the model's final performance across all tasks after the entire training sequence has been completed. It is calculated by taking the arithmetic mean of the performance on every task j after the model has finished training on the final task.

$$AmAP = \frac{1}{T} \sum_{j=1}^T R_{T,j} \quad (4.6)$$

A high AmAP indicates strong overall knowledge retention and generalization at the end of the learning process [30].

4.4.2 Average Incremental mAP

AImAP gives a broader assessment of performance throughout the entire learning process, rather than just at the final stage. For each task T_i in the sequence, it calculates the average performance on all tasks seen up to that point (i.e. tasks T_1 to T_i). The final metric is the average of these values over all T tasks.

$$AImAP = \frac{1}{T} \sum_{i=1}^T \left(\frac{1}{i} \sum_{j=1}^i R_{i,j} \right) \quad (4.7)$$

4 Dataset and Experimental Setup

In contrast to AmAP, AImAP further reflects the historical performance of the model [30].

4.4.3 Forgetting

Forgetting aims to quantify the degree of catastrophic forgetting. For any given task T_j , Forgetting is measured as the difference between the peak performance achieved on that task at any point during training and the final performance on that same task after all T tasks have been learned. The overall Forgetting is the average of this value across all but the final task [39].

$$Forgetting = \frac{1}{T-1} \sum_{j=1}^{T-1} \left(\max_{k \in \{j, \dots, T-1\}} R_{k,j} - R_{T,j} \right) \quad (4.8)$$

A lower Forgetting value is desirable, as it indicates that the model successfully retains knowledge from earlier tasks.

5 Experiments and Results

This chapter presents the empirical results of the investigation into the suitability of LoRA adapter merging as a paradigm for CL. The experimental process begins with the evaluation of baseline performance, which serve as a reference point for subsequent comparisons. Following this, a comparative analysis of the three merging strategies – Weight Averaging, TIES-Merging, and CAT – is conducted to evaluate their effectiveness in a CL setting. In addition to the original CAT formulation, an adapted variant, LinCAT, is tested to address the scalability limitations caused by CAT’s exponential parameter growth. Subsequently the effect of the LoRA rank is examined to determine whether increasing the rank, and thus the size of the adapters, leads to a discernible improvement in performance and how adapter size may impact the efficacy of techniques like TIES-merging. Lastly, one experiment is dedicated to the TIES-Merging strategy, exploring how different keep-values k , affect its performance.

5.1 Baseline Performance

In Figure 5.1 catastrophic forgetting is depicted by the gap between the lower and upper baseline.

The performance of the lower baseline illustrates the combined impact of forgetting and the varying characteristics of the sequential tasks. This degradation of knowledge is quantified over the entire sequence by a high Forgetting score of 0.2415, as seen in Table 5.2. The fluctuations, particularly the noticeable performance peaks at Finland (T1) and Lithuania (T4), indicate that the learning dynamics are more complex than can be explained by forgetting alone. Some insight into this behavior is provided by Table 5.1, which shows the model’s performance on each country’s validation set after being trained only on Finland (T1). The model achieves a surprisingly high mAP of 0.5844 on the unseen Lithuanian data, surpassing its performance on the other unseen tasks. This suggests a high degree of knowledge transferability between the Finnish and Lithuanian datasets. This underlying similarity offers two potential explanations for the performance spike at T4 in Figure 5.1. First, the knowledge retained from previous tasks (especially Finland) may create a forward transfer of knowledge, that improves learning on the similar Lithuanian task. Second, it is possible that the land-cover distributions in both Finland and Lithuania contain features that are inherently easier for the model to classify, leading to higher mAP scores. Therefore, the lower baseline’s trajectory should be interpreted as a complex play between the degradation of knowledge due to forgetting and the varying difficulty and transferability of the tasks themselves.

5 Experiments and Results

Table 5.1: Validation mAP (micro) after training only on Task 1 (Finland), evaluated on each single-country’s validation set.

Evaluation country	mAP (micro) $\uparrow$
Finland	0.9107
Portugal	0.3262
Serbia	0.3632
Lithuania	0.5844
Ireland	0.3741
Austria	0.5232

As shown in Figure 5.1, the upper baseline maintains a high and stable mAP across all tasks, with a slight upward trend as the cumulative training data grows. Table 5.2 supports this observation with a near-zero Forgetting score 0.0008 and a high AmAP of 0.8380. Together, these two baselines frame the challenge for the merging strategies: to approach the performance of the upper baseline while adhering to the constraint of the lower baseline.

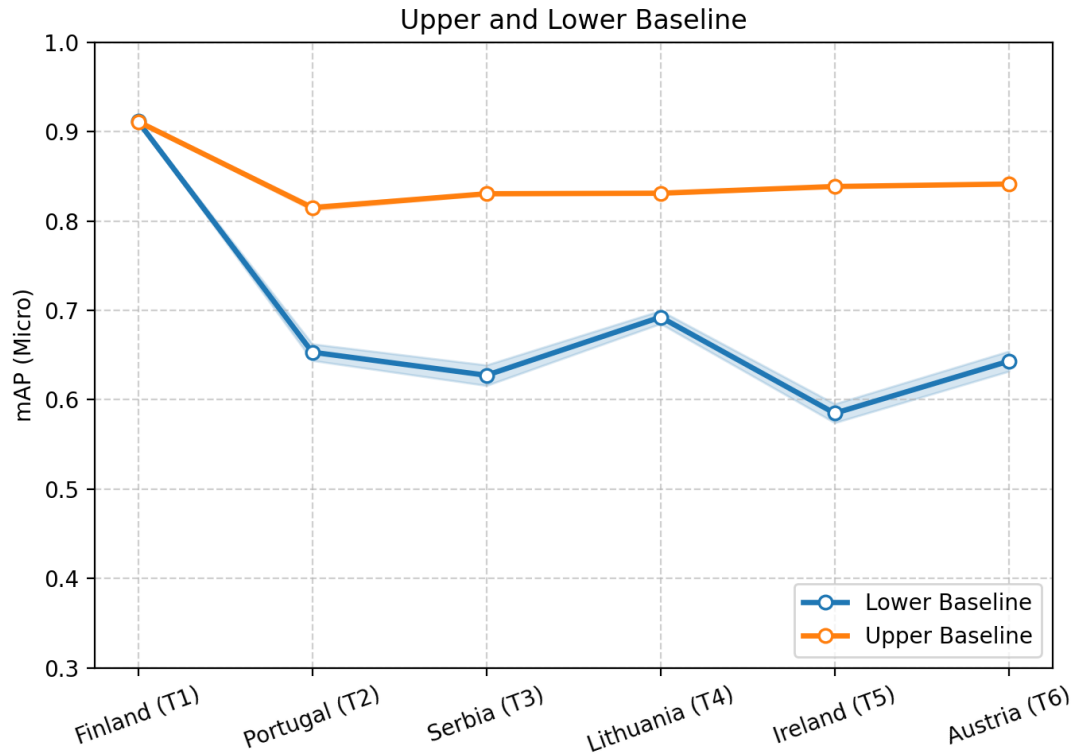


Figure 5.1: This plot serves as a visualization of the baselines performance. For each step t (x-axis), the plot shows the AmAP achieved after training on task T_t (y-axis).

Table 5.2: Metrics (mean $\pm$ std over 5 runs) for lower and upper baselines.

Metric	Lower	Upper
AmAP (micro) $\uparrow$	0.6386 ± 0.0132	0.8380 ± 0.0015
AImAP (micro) $\uparrow$	0.6837 ± 0.0048	0.8412 ± 0.0013
Forgetting (micro) $\downarrow$	0.2415 ± 0.0162	0.0008 ± 0.0020
AmAP (macro) $\uparrow$	0.4316 ± 0.0043	0.5123 ± 0.0029
AImAP (macro) $\uparrow$	0.4384 ± 0.0041	0.4960 ± 0.0024
Forgetting (macro) $\downarrow$	0.0970 ± 0.0021	-0.0017 ± 0.0030

5.2 Comparison of Merging Strategies

The empirical results, illustrated in Figure 5.2 and detailed in Table 5.3, show that all the evaluated merging methods successfully improve upon the the lower baseline. Each method demonstrates a significant reduction in catastrophic forgetting, as evidenced by their substantially lower Forgetting metrics compared to the baseline’s score of 0.2415. The standard deviation across all merging strategies is consistently low, which indicates that the frozen backbone’s provided representations, lead to robust and reproducible results. Furthermore the performance fluctuations observed in the lower baseline are partially smoothed out.

Weight Averaging and TIES-Merging exhibit similar performance trajectories. Both strategies clearly counteract the effects of catastrophic forgetting. However, a closer look at the metrics in Table 5.3 reveals that TIES achieves a lower Forgetting score (0.0500 ± 0.0047) than Weight Averaging (0.0906 ± 0.0041). This suggests that Weight Averaging is more susceptible to catastrophic forgetting, a weakness that could become more pronounced in scenarios with more than six sequential tasks. It is noteworthy that the overall performance of TIES plateaus around the level of Weight Averaging instead of surpassing it more decisively. One potential explanation is that the task-specific adapters have minimal parameter sign interference among them; this would nullify the primary advantage of TIES, which is designed to resolve such conflicts. Alternatively, the merging process, which always combines only two adapters at a time, may limit the effectiveness of the sign-election mechanism. The benefits of TIES might become more apparent in a setup where more than two adapters are merged simultaneously. Other factors, such as the LoRA rank r or the keep-value k , could also play a role in this observation. Their effects are explored in chapters 5.5 and 5.5.

Among the evaluated methods, CAT emerges as the best-performing strategy. It surpasses both Weight Averaging and TIES-Merging in the AmAP, AImAP and Forgetting metrics. Despite its superior performance, this implementation of CAT has a significant drawback that limits its feasibility for CL applications with a large number of tasks. As CAT concatenates adapters at each step, the number of parameters in the LoRA module being trained at task t , denoted as $|\theta_t^{LoRA}|$, scales exponentially with the number of tasks.

$$|\theta_t^{LoRA}| = |\theta_1^{LoRA}| \cdot 2^{t-2}, \quad \text{for } t \geq 2 \quad (5.1)$$

5 Experiments and Results

For the LoRA parameters and backbone model used in this experiment, this exponential growth means that by the ninth task, the newly created LoRA adapter would contain more parameters than the entire frozen backbone. This scalability issue renders this version of CAT impractical for open-ended CL scenarios, restricting its use to applications with a predefined, small number of sequential learning steps. It is also noteworthy that, as the merging process incorporates a fine-tuning step, it necessitates retaining a small subset of the training data from all previously encountered tasks.

Table 5.3: Metrics (mean $\pm$ std over 5 runs) for baselines and merging strategies.

Metric	Lower	Weight Averaging	TIES	CAT	Upper
AmAP (micro) $\uparrow$	0.6386 $\pm$ 0.0132	0.7167 $\pm$ 0.0028	0.7279 $\pm$ 0.0050	0.7774 $\pm$ 0.0018	0.8380 $\pm$ 0.0015
AIImAP (micro) $\uparrow$	0.6837 $\pm$ 0.0048	0.7432 $\pm$ 0.0031	0.7481 $\pm$ 0.0045	0.7995 $\pm$ 0.0011	0.8412 $\pm$ 0.0013
Forgetting (micro) $\downarrow$	0.2415 $\pm$ 0.0162	0.0906 $\pm$ 0.0041	0.0500 $\pm$ 0.0047	0.0380 $\pm$ 0.0032	0.0008 $\pm$ 0.0020
AmAP (macro) $\uparrow$	0.4316 $\pm$ 0.0043	0.4543 $\pm$ 0.0032	0.4519 $\pm$ 0.0022	0.4504 $\pm$ 0.0038	0.5123 $\pm$ 0.0029
AIImAP (macro) $\uparrow$	0.4384 $\pm$ 0.0041	0.4526 $\pm$ 0.0033	0.4514 $\pm$ 0.0046	0.4565 $\pm$ 0.0027	0.4960 $\pm$ 0.0024
Forgetting (macro) $\downarrow$	0.0970 $\pm$ 0.0021	0.0270 $\pm$ 0.0073	0.0159 $\pm$ 0.0063	0.0345 $\pm$ 0.0043	-0.0017 $\pm$ 0.0030

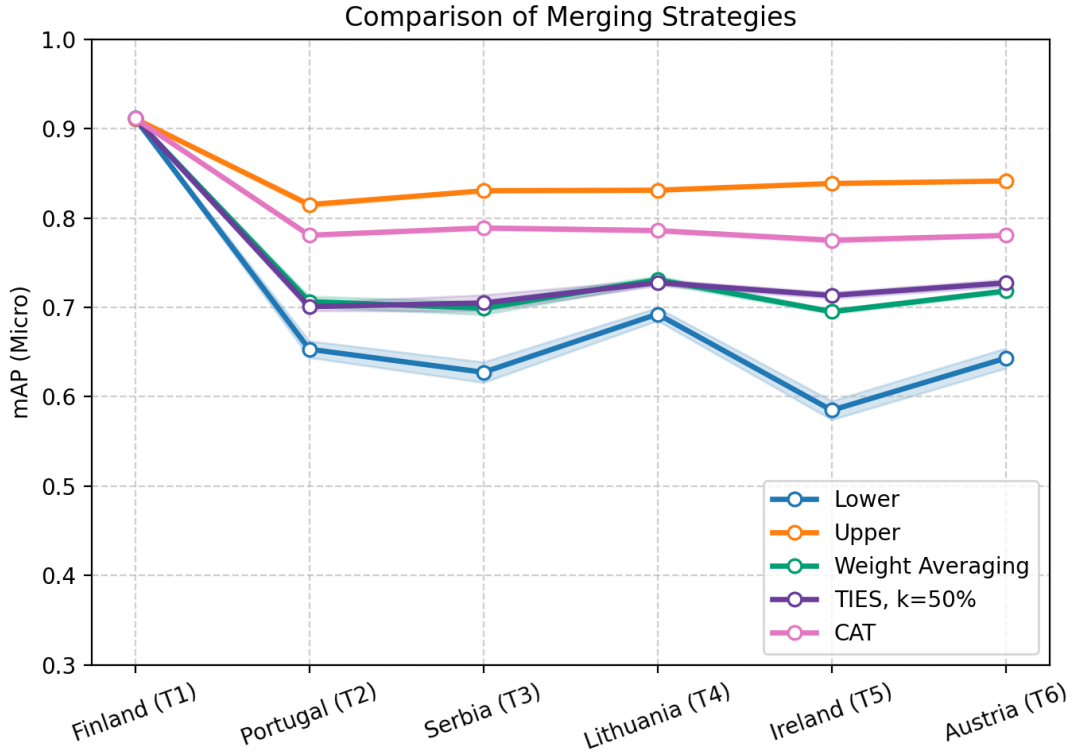


Figure 5.2: This plot visualizes the comparison of merging strategies. For each step t (x-axis), the plot shows the AmAP achieved after training on task T_t (y-axis) for each of the merging strategies.

5.3 Controlling Parameter Growth in CAT

While the CAT-Merging strategy demonstrated superior performance, its exponential growth in parameters, as outlined in the previous section, renders it impractical for CL scenarios with a large or undefined number of tasks. To address this scalability issue, a modified approach, hereafter referred to as LinCAT, is examined. This adaptation of the previous CAT method, alters the training and merging pipeline to prevent the exponential increase in adapter size. In the LinCAT pipeline, each new task-specific adapter is trained independently. This ensures that the number of trainable parameters during the adapter training phase for each task remains constant. The integration of the task-specific knowledge occurs exclusively during the merging step.

The process begins by training a standard LoRA adapter, θ_1^* , on the first task. For every subsequent task $t \geq 2$, the pipeline operates in two stages. First, a fresh, LoRA adapter, θ_t' , is trained independently on the new task data, D_t , starting from the initial parameters θ_0 .

$$\theta_t' = \text{Train}(\theta_0, D_t) \quad (5.2)$$

Next, this newly trained adapter is merged with the consolidated adapter from the previous step, θ_{t-1}^* . The CAT methodology concatenates their LoRA matrices to form the new consolidated model, θ_t^* .

$$\theta_t^* = \text{Merge}_{\text{CAT}}(\theta_{t-1}^*, \theta_t') \quad (5.3)$$

This architectural change is the source of the linear scalability. The rank of the consolidated adapter increases by a constant value, r , at each step, leading to a cumulative rank of tr at task t . Consequently, the total number of parameters in the merged LoRA module grows linearly with the number of tasks. This stands in stark contrast to the previous CAT implementation, where the rank – and thus the parameter count – doubled at each step, leading to exponential growth. The empirical results, shown in Figure 5.3 and Table 5.4, validate this approach. While LinCAT does not reach the peak performance of the exponential scaling implementation of CAT, it consistently and significantly outperforms both Weight Averaging and TIES-Merging. This demonstrates that LinCAT offers a compelling trade-off, sacrificing a small amount of performance for a large gain in scalability, making it a more viable solution for real-world CL applications.

Table 5.4: Metrics (mean $\pm$ std over 5 runs) for comparing LinCAT against the other previously established methods.

Metric	Weight Averaging	TIES	LinCAT	CAT
AmAP (micro) $\uparrow$	0.7167 ± 0.0028	0.7279 ± 0.0050	0.7576 ± 0.0030	0.7774 ± 0.0018
AImAP (micro) $\uparrow$	0.7432 ± 0.0031	0.7481 ± 0.0045	0.7806 ± 0.0014	0.7995 ± 0.0011
Forgetting (micro) $\downarrow$	0.0906 ± 0.0041	0.0500 ± 0.0047	0.0418 ± 0.0015	0.0380 ± 0.0032
AmAP (macro) $\uparrow$	0.4543 ± 0.0032	0.4519 ± 0.0022	0.4211 ± 0.0030	0.4504 ± 0.0038
AImAP (macro) $\uparrow$	0.4526 ± 0.0033	0.4514 ± 0.0046	0.4284 ± 0.0022	0.4565 ± 0.0027
Forgetting (macro) $\downarrow$	0.0270 ± 0.0073	0.0159 ± 0.0063	0.0383 ± 0.0046	0.0345 ± 0.0043

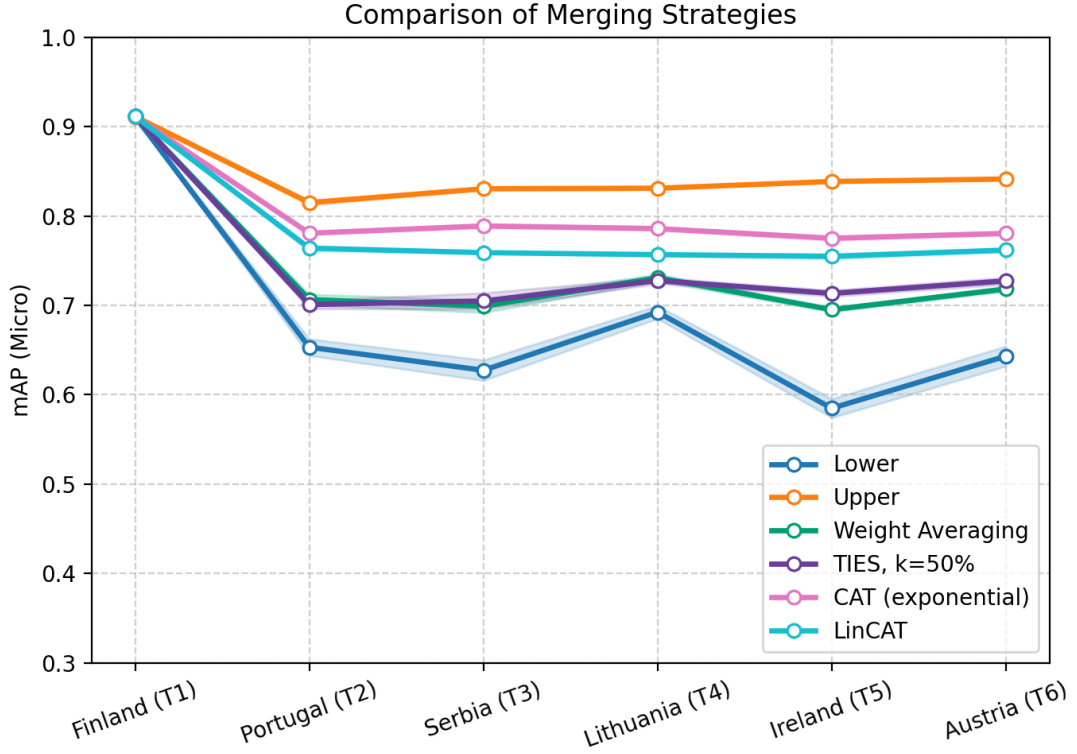


Figure 5.3: A comparison of merging strategies including LinCAT. For each step t (x-axis), the plot shows the AmAP achieved after training on task T_t (y-axis).

5.4 Effect of LoRA Rank

This section examines whether increasing the LoRA rank and consequently the adapter size, leads to improved performance. For this investigation, LinCAT is utilized, as the exponential growth in parameters of the original CAT version, showcased in section 5.2, would be computationally prohibitive at higher ranks.

The plots in Figure 5.4 reveal that LinCAT’s performance is only slightly affected by an increase in LoRA rank. While there are minor gains in both AmAP and AImAP as the rank increases, the Forgetting metric appears to plateau from rank 16 onwards, showing no further improvement. In contrast to LinCAT, Weight Averaging and TIES-Merging exhibit very similar behavior to each other as the LoRA rank is increased. Both methods show symmetrical climb in performance that gradually tapers off at higher ranks. This outcome suggests that increasing the number of parameters does not necessarily lead to more significant sign interference between the adapters. Since the primary advantage of TIES-Merging lies in resolving such conflicts, its inability to significantly outperform Weight Averaging can be attributed to the potential lack of substantial interference to mitigate. Therefore, the conclusions drawn in section 5.2 remain valid: there may be minimal sign interference between the adapters in this setup, and the true

5.4 Effect of LoRA Rank

benefits of TIES might be more pronounced in a different merging scenario, such as combining more than two adapters at once.

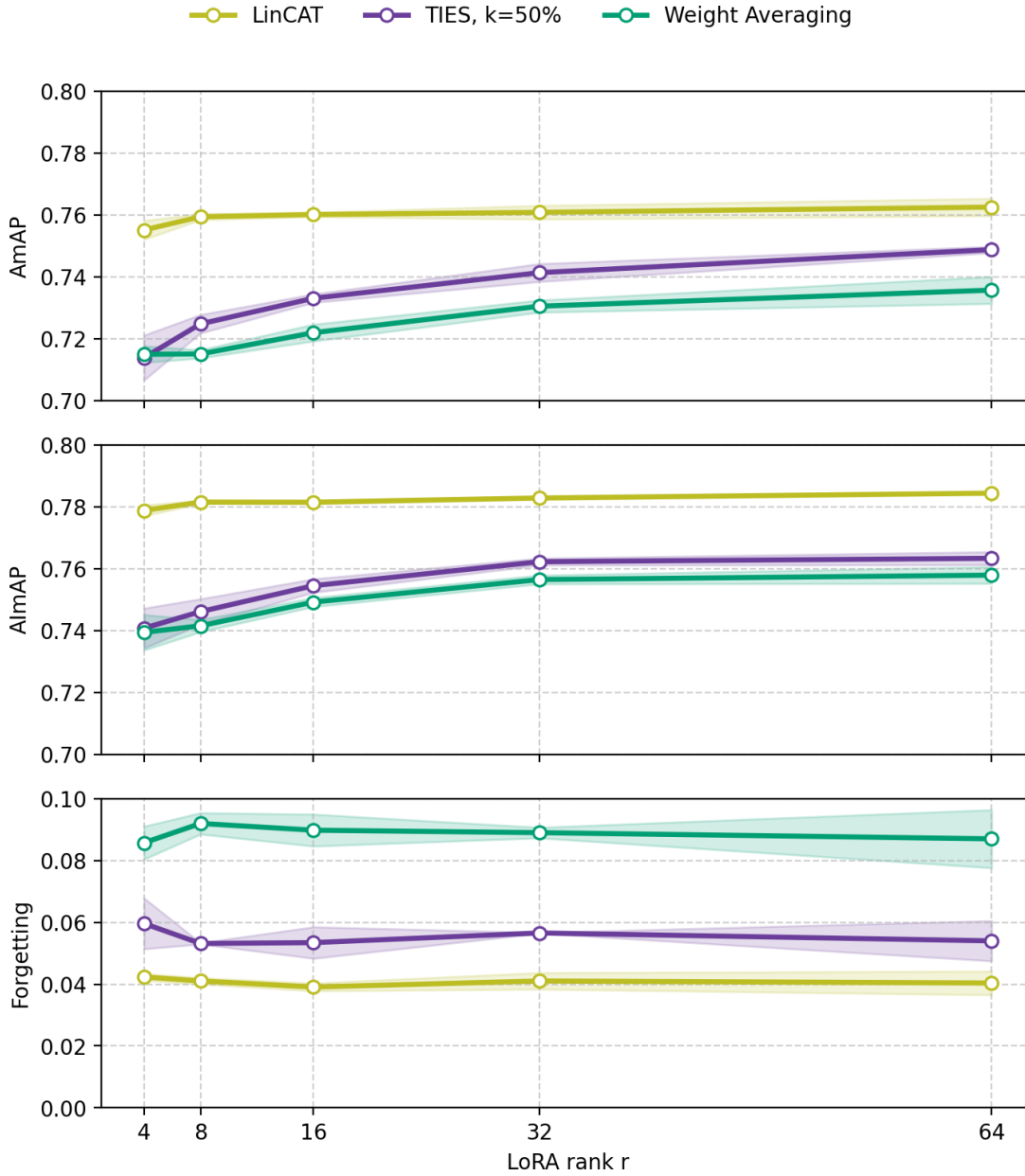


Figure 5.4: Illustration for the impact of the LoRA rank hyperparameter. The x-axis displays the LoRA rank. The y-axis displays the corresponding metric score for each of the merging methods, evaluated over 3 runs.

5.5 Effect of TIES-Merging Keep-Values

This section investigates the influence of the keep-value hyperparameter, k , which is central to the Trim stage of the TIES-Merging algorithm. The keep-value determines the percentage of weights with the highest absolute magnitude to be retained, while the rest are pruned by setting them to zero (see Chapter 3.3.2).

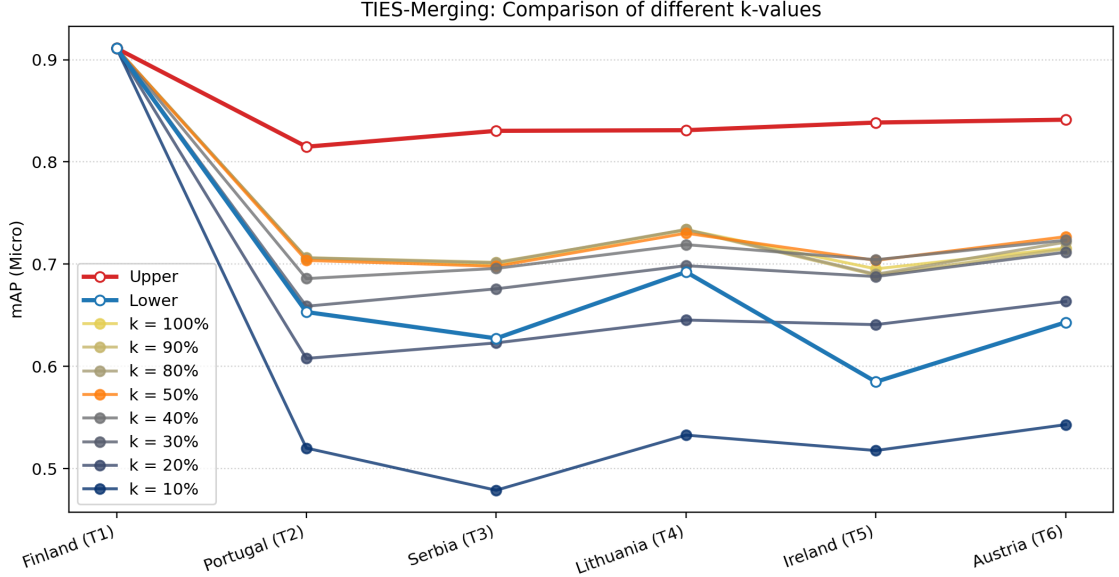


Figure 5.5: A comparison of multiple TIES keep-values, k . For each step t (x-axis), the plot shows the AmAP achieved after training on task T_t (y-axis). This is done for all the different k -values, seen in the legend.

As depicted in Figure 5.5, the performance of TIES exhibits a clear dependency on k . There is a steady improvement in mAP as k increases from 10% to 50%. However, beyond this point, a performance plateau is observed, with the results for $k = 50\%$, 80% , 90% and 100% being nearly indistinguishable. This presents a notable deviation from the results reported in [21], where peak performance was often achieved with lower keep-values, such as $k = 20\%$. A likely explanation for this discrepancy lies in the nature of the LoRA adapters being merged. The original work focused on fully fine-tuned models, which often contain a high degree of parameter redundancy. In contrast, the experiments in this thesis apply TIES-Merging to LoRA adapters, which are inherently parameter-efficient. The already compact nature of LoRA suggests that a higher proportion of its weights are critical for task performance. Even so, the clear plateau at $k = 50\%$ is a significant observation. It implies that approximately half of the weights within the LoRA adapters have magnitudes too small to meaningfully contribute to the final merged model’s performance.

It is also important to note a practical aspect of the implementation. The Trim process sets redundant parameters to zero but does not alter the underlying architecture or reduce the total parameter count for inference. Therefore, in the case of the experiments conducted in this work,

5.5 *Effect of TIES-Merging Keep-Values*

the choice of k should not impact computational efficiency or memory footprint. Based on these findings, a keep-value of $k = 50\%$ is selected for all other experiments involving TIES-Merging in this thesis. This value is showcasing the algorithm's capability of leveraging only the most critical half of the available parameters, while still achieving peak performance.

6 Conclusion and Future Work

This thesis investigated the suitability of LoRA adapter merging in the context of CL for EO applications. Applying this technique to a pre-trained FM, in a DIL multi-label classification setting based on Sentinel-2 imagery, the empirical results demonstrate that merging LoRA adapters is a viable strategy for mitigating catastrophic forgetting. All evaluated merging methods – Weight Averaging, TIES-Merging, and CAT, including its linear scaling variant LinCAT – consistently improved performance compared to the lower baseline. While CAT achieved the highest overall performance, its exponential growth in parameters limits its practicality. LinCAT, in contrast, offers a favorable trade-off by approaching CAT-level performance while maintaining linear parameter growth, making it the most attractive merging strategy under realistic resource constraints.

Overall, this work (i) establishes a LoRA-based merging pipeline for a pre-trained EO FM in a realistic CL setting, (ii) provides a systematic comparison of multiple adapter merging strategies, and (iii) demonstrates that appropriately merged LoRA adapters can reduce catastrophic forgetting while remaining parameter-efficient.

The insights gained from this thesis open several avenues for future research, including:

1. **Benchmarking against Established CL Paradigms:** A crucial next step is to benchmark the LoRA merging approach against other established CL paradigms in the RS domain (see Chapter 3.2). This would provide a more comprehensive understanding of its comparative strengths and weaknesses.
2. **Long-Term Performance Evaluation:** The current experiments were conducted with a sequence of six tasks. Future work should involve testing these merging strategies on a significantly larger number of tasks to evaluate their performance and scalability under long-term CL conditions.
3. **CIL Setup:** As this thesis focused on a DIL setup, investigating the performance of LoRA merging in a CIL setup would deliver valuable insights. CIL poses a different challenge, requiring models to learn new classes without forgetting old ones.
4. **Impact of Larger Datasets:** The experiments used 10,000 samples per task. While it is reasonable to hypothesize that training adapters on larger datasets would result in better performance, it would be particularly interesting to see its effect on the mAP-Macro scores. An increase in the number of samples, especially for rarer classes, could lead to a significant improvement in macro performance.
5. **Training on a Frozen Head:** During implementation, there were indications that similar

performance could be achieved with a frozen classification head, eliminating the necessity for an independent head merging strategy.

6. **Influence of Task Order:** The current experimental setup used a fixed task order. Future work could explore the influence of different task orders on the performance of the merging strategies.
7. **Handling Diverse Task Vectors:** The tasks in this thesis showed signs of having similar task vector directions. It would be insightful to evaluate how the methods, especially TIES-Merging designed to resolve parameter interference, perform when confronted with highly dissimilar tasks.
8. **Alternative Learning Tasks:** This work focused on multi-label classification. Investigating how the approach performs under different learning tasks, such as segmentation or object detection, would demonstrate the broader applicability of these strategies in RS.

Bibliography

- [1] D.-W. Zhou et al. *Continual Learning with Pre-Trained Models: A Survey*. 2024. doi: 10.48550/arXiv.2401.16386. arXiv: 2401.16386. URL: <http://arxiv.org/abs/2401.16386> (visited on 11/03/2025).
- [2] Y. Lee et al. “ER-PASS: Experience Replay with Performance-Aware Submodular Sampling for Domain-Incremental Learning in Remote Sensing”. In: *Remote Sensing* 17.18 (2025), p. 3233. doi: 10.3390/rs17183233. URL: <https://www.mdpi.com/2072-4292/17/18/3233> (visited on 11/03/2025).
- [3] N. Díaz-Rodríguez et al. *Don’t forget, there is more than forgetting: new metrics for Continual Learning*. 2018. doi: 10.48550/arXiv.1810.13166. arXiv: 1810.13166. URL: <http://arxiv.org/abs/1810.13166> (visited on 11/03/2025).
- [4] H. Li et al. “CLRS: Continual Learning Benchmark for Remote Sensing Image Scene Classification”. In: *Sensors* 20.4 (2020), p. 1226. doi: 10.3390/s20041226. URL: <https://www.mdpi.com/1424-8220/20/4/1226> (visited on 11/03/2025).
- [5] M.-A. Boum et al. “Continual Learning in Remote Sensing: Leveraging Foundation Models and Generative Classifiers to Mitigate Forgetting”. In: *IGARSS 2024 - 2024 IEEE International Geoscience and Remote Sensing Symposium*. 2024, pp. 8535–8540. doi: 10.1109/IGARSS53475.2024.10640659. URL: <https://ieeexplore.ieee.org/document/10640659> (visited on 11/03/2025).
- [6] P. Janson et al. *A Simple Baseline that Questions the Use of Pretrained-Models in Continual Learning*. 2023. doi: 10.48550/arXiv.2210.04428. arXiv: 2210.04428. URL: <http://arxiv.org/abs/2210.04428> (visited on 11/03/2025).
- [7] Q. Gao et al. *A Unified Continual Learning Framework with General Parameter-Efficient Tuning*. 2023. doi: 10.48550/arXiv.2303.10070. arXiv: 2303.10070. URL: <http://arxiv.org/abs/2303.10070> (visited on 11/03/2025).
- [8] X. Zhang et al. *C-LoRA: Continual Low-Rank Adaptation for Pre-trained Models*. 2025. doi: 10.48550/arXiv.2502.17920. arXiv: 2502.17920. URL: <http://arxiv.org/abs/2502.17920> (visited on 11/03/2025).
- [9] E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. doi: 10.48550/arXiv.2106.09685. arXiv: 2106.09685. URL: <http://arxiv.org/abs/2106.09685> (visited on 11/03/2025).
- [10] A. Prabhakar et al. *LoRA Soups: Merging LoRAs for Practical Skill Composition Tasks*. 2024. doi: 10.48550/arXiv.2410.13025. arXiv: 2410.13025. URL: <http://arxiv.org/abs/2410.13025> (visited on 11/03/2025).

- [11] L. Zhao et al. “Continual Learning for Remote Sensing Image Scene Classification With Prompt Learning”. In: *IEEE Geoscience and Remote Sensing Letters* (2023), pp. 1–5. doi: 10.1109/LGRS.2023.3328981. URL: <https://ieeexplore.ieee.org/document/10304174> (visited on 11/03/2025).
- [12] R. Peng et al. “Continual Contrastive Learning for Cross-Dataset Scene Classification”. In: *Remote Sensing* 14 (2022), p. 5105. doi: 10.3390/rs14205105.
- [13] L. Möllenbrok et al. “Continual Self-Supervised Learning with Masked Autoencoders in Remote Sensing”. In: *IEEE Geoscience and Remote Sensing Letters* 22 (2025), pp. 1–5. doi: 10.1109/LGRS.2025.3579585. URL: <http://arxiv.org/abs/2506.21312> (visited on 11/03/2025).
- [14] A. S. Alakooz and N. Ammour. “A Contrastive Continual Learning for the Classification of Remote Sensing Imagery”. In: *IGARSS 2022 - 2022 IEEE International Geoscience and Remote Sensing Symposium*. 2022, pp. 7902–7905. doi: 10.1109/IGARSS46834.2022.9884527. URL: <https://ieeexplore.ieee.org/document/9884527> (visited on 11/03/2025).
- [15] A. Marvellous et al. *Continual Learning Approaches to Retain Segmentation Performance Across Seasonal Satellite Images*. 2025.
- [16] S. D. Bhat et al. *Efficient Curriculum based Continual Learning with Informative Subset Selection for Remote Sensing Scene Classification*. 2023. doi: 10.48550/arXiv.2309.01050. arXiv: 2309.01050. URL: <http://arxiv.org/abs/2309.01050> (visited on 11/03/2025).
- [17] V. Marsocci and S. Scardapane. “Continual Barlow Twins: Continual Self-Supervised Learning for Remote Sensing Semantic Segmentation”. In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 16 (2023), pp. 5049–5060. doi: 10.1109/JSTARS.2023.3280029. URL: <https://ieeexplore.ieee.org/document/10135093> (visited on 11/03/2025).
- [18] M. Wang et al. “Domain-incremental learning for fire detection in space-air-ground integrated observation network”. In: *International Journal of Applied Earth Observation and Geoinformation* 118 (2023), p. 103279. doi: 10.1016/j.jag.2023.103279. URL: <https://www.sciencedirect.com/science/article/pii/S1569843223001012> (visited on 11/03/2025).
- [19] J. Peng et al. “Asymmetric Collaborative Network: Transferable Lifelong Learning for Remote Sensing Images”. In: *IGARSS 2022 - 2022 IEEE International Geoscience and Remote Sensing Symposium*. 2022, pp. 5057–5060. doi: 10.1109/IGARSS46834.2022.9884708. URL: <https://ieeexplore.ieee.org/document/9884708> (visited on 11/03/2025).
- [20] W. Ruan et al. *From Task-Specific Models to Unified Systems: A Review of Model Merging Approaches*. 2025. doi: 10.48550/arXiv.2503.08998. arXiv: 2503.08998. URL: <http://arxiv.org/abs/2503.08998> (visited on 11/03/2025).
- [21] P. Yadav et al. “TIES-Merging: Resolving Interference When Merging Models”. In: 2023. URL: <https://openreview.net/forum?id=xtaX3WyCj1> (visited on 11/03/2025).

Bibliography

- [22] M. Wortsman et al. *Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time*. 2022. doi: 10.48550/arXiv.2203.05482. arXiv: 2203.05482. URL: <http://arxiv.org/abs/2203.05482> (visited on 11/03/2025).
- [23] L. Choshen et al. *Fusing finetuned models for better pretraining*. 2022. doi: 10.48550/arXiv.2204.03044. arXiv: 2204.03044. URL: <http://arxiv.org/abs/2204.03044> (visited on 11/03/2025).
- [24] K. Wang et al. *Localizing Task Information for Improved Model Merging and Compression*. 2024. doi: 10.48550/arXiv.2405.07813. arXiv: 2405.07813. URL: <http://arxiv.org/abs/2405.07813> (visited on 11/15/2025).
- [25] S. Zheng and H. Wang. *FREE-Merging: Fourier Transform for Efficient Model Merging*. 2024. doi: 10.48550/arXiv.2411.16815. arXiv: 2411.16815. URL: <http://arxiv.org/abs/2411.16815> (visited on 11/15/2025).
- [26] M. De Lange et al. “A continual learning survey: Defying forgetting in classification tasks”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2021). doi: 10.1109/TPAMI.2021.3057446. URL: <http://arxiv.org/abs/1909.08383> (visited on 11/03/2025).
- [27] D.-W. Zhou et al. “Class-Incremental Learning: A Survey”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.12 (2024), pp. 9851–9873. doi: 10.1109/TPAMI.2024.3429383. URL: <http://arxiv.org/abs/2302.03648> (visited on 11/03/2025).
- [28] H. Shi and H. Wang. *A Unified Approach to Domain Incremental Learning with Memory: Theory and Algorithm*. 2023. doi: 10.48550/arXiv.2310.12244. arXiv: 2310.12244. URL: <http://arxiv.org/abs/2310.12244> (visited on 11/03/2025).
- [29] Y. Wu et al. *SD-LoRA: Scalable Decoupled Low-Rank Adaptation for Class Incremental Learning*. 2025. doi: 10.48550/arXiv.2501.13198. arXiv: 2501.13198. URL: <http://arxiv.org/abs/2501.13198> (visited on 11/03/2025).
- [30] L. Wang et al. *A Comprehensive Survey of Continual Learning: Theory, Method and Application*. 2024. doi: 10.48550/arXiv.2302.00487. arXiv: 2302.00487. URL: <http://arxiv.org/abs/2302.00487> (visited on 11/07/2025).
- [31] A. Aghajanyan et al. *Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning*. 2020. doi: 10.48550/arXiv.2012.13255. arXiv: 2012.13255. URL: <http://arxiv.org/abs/2012.13255> (visited on 11/03/2025).
- [32] D. Tang et al. “LoRA Merging with SVD: Understanding Interference and Preserving Performance”. In: 2025. URL: <https://openreview.net/forum?id=t9FrMviTaP> (visited on 11/03/2025).
- [33] K. N. Clasen et al. *reBEN: Refined BigEarthNet Dataset for Remote Sensing Image Analysis*. 2025. doi: 10.48550/arXiv.2407.03653. arXiv: 2407.03653. URL: <http://arxiv.org/abs/2407.03653> (visited on 11/04/2025).

- [34] Y. Wang et al. *Towards a Unified Copernicus Foundation Model for Earth Vision*. 2025. doi: 10.48550/arXiv.2503.11849. arXiv: 2503.11849. URL: <http://arxiv.org/abs/2503.11849> (visited on 11/05/2025).
- [35] L. Hackel et al. “ConfigILM: A General Purpose Configurable Library for Combining Image and Language Models for Visual Question Answering”. In: *SoftwareX* 26 (2024), p. 101731. doi: 10.1016/j.softx.2024.101731. URL: <https://www.sciencedirect.com/science/article/pii/S235271102400102X> (visited on 11/03/2025).
- [36] D. Bolya et al. *TIDE: A General Toolbox for Identifying Object Detection Errors*. 2020. doi: 10.48550/arXiv.2008.08115. arXiv: 2008.08115. URL: <http://arxiv.org/abs/2008.08115> (visited on 11/07/2025).
- [37] M. Sokolova and G. Lapalme. “A Systematic Analysis of Performance Measures for Classification Tasks”. In: *Information Processing & Management* 45.4 (2009), pp. 427–437. doi: 10.1016/j.ipm.2009.03.002. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0306457309000259> (visited on 11/07/2025).
- [38] J. Opitz. “A Closer Look at Classification Evaluation Metrics and a Critical Reflection of Common Evaluation Practice”. In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 820–836. doi: 10.1162/tac1_a_00675. URL: https://doi.org/10.1162/tac1_a_00675 (visited on 11/08/2025).
- [39] A. Chaudhry et al. “Riemannian Walk for Incremental Learning: Understanding Forgetting and Intransigence”. In: *Computer Vision – ECCV 2018*. Vol. 11215. Lecture Notes in Computer Science. 2018, pp. 556–572. doi: 10.1007/978-3-030-01252-6_33. URL: <http://arxiv.org/abs/1801.10112> (visited on 11/07/2025).